VLA 기술 문서

ROS 2 Jazzy · MoveIt 2 · Ollama · 탑뷰 RGB + ArUco 호모그래피 (시뮬레이션 단계)

2026-10-15

1. 개요

1.1 목표

자연어 명령(한국어/영어)을 받아 탑뷰 RGB 카메라로 테이블 위 색 블록을 인식하고, LLM이 작업을 결정하면 ROBOTIS OMX-F가 MoveIt 2로 top-down pick & place를 수행한다. Depth 센서는 없고, 테이블 네 꼭짓점의 ArUco 마커로 이미지 평면과 테이블 평면 사이의 호모그래피를 구한다.

1.2 접근 방식: 모듈형 VLA

End-to-end 학습 정책 대신 **VLM/LLM 플래너 + 고전 인식 + 모션 플래닝** 구조를 택했다.

구성	담당	학습 데이터
Vision	OpenCV: ArUco, 호모그래피, HSV 색 분할	불필요
Language	Ollama `gemma4:e2b` : 명령 → 심볼릭 계획(JSON)	불필요 (zero-shot)
Action	닫힌형 top-down IK + MoveIt 2 (Pilz PTP)	불필요

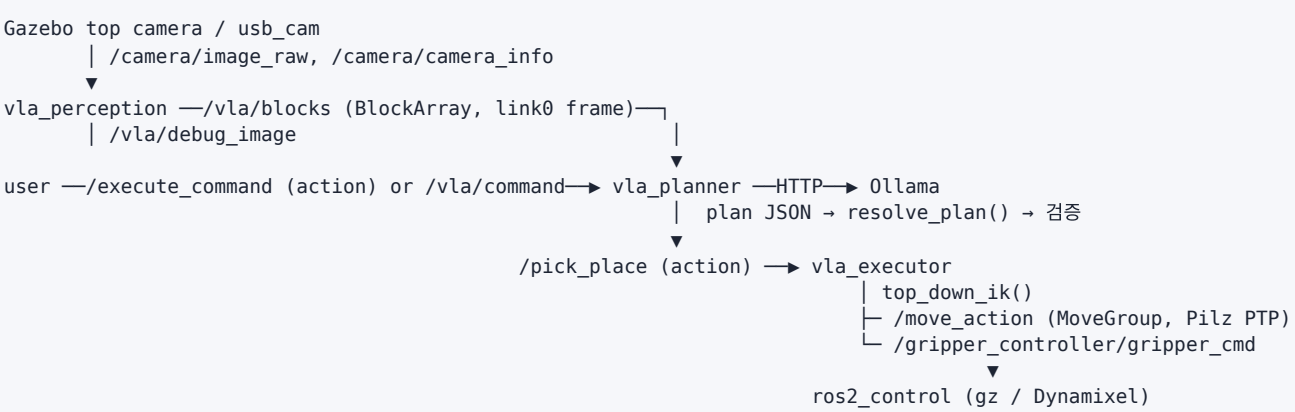
핵심 원칙: **LLM은 좌표를 생성하지 않는다.** 검출된 블록 ID와 허용된 장소 토큰 중에서만 고르고, 미터 단위 좌표는 결정적인 코드가 계산·검증한다. 2B급 소형 모델의 수치 환각을 구조적으로 막는다.

1.3 현재 상태 요약

항목	결과
블록 위치 추정 (Gazebo GT 대비)	최대 2.1 mm, yaw 오차 0°
카메라 위치 역산	(0.19, 0.00, 0.75) m (설정값과 일치)
관절 추종 오차 (게인 수정 후)	< 0.001 rad
좌표 지정 이동, 3회 반복	3/3, 4.3–5.4 mm
23° 회전 블록 집기, 3회 반복	3/3
쌓기, 반복	8/9 (회전 블록 포함, 놓기 개선 후)
손목 카메라 집기 확인	정상 85–99% 통과 / 놓침 4종 시나리오 0%로 실패 판정
LLM 명령 해석 (10개)	10/10, 평균 3.1 s (cold start ≈15 s 포함)
최종 end-to-end (5개)	5/5 (실행 4, 올바른 거부 1), 오차 3.5–12.6 mm
캘리브레이션 도구 (시뮬 검증)	intrinsics fx +0.03 % · colors 재현율 98–100 % · table 2° 회전 오차 복원(−2.04°), 잔차 3.1 mm · wrist 여유 89 %
단위 테스트	43 passed (planner 26, perception 4, executor 5, calibration 8)

2. 시스템 구성

2.1 데이터 흐름



2.2 패키지

패키지	빌드	주요 파일	역할
`open_manipulator` (ROBOTIS, jazzy)	ament	`omx_f.urdf.xacro`, `omx_f_moveit.launch.py`	URDF, ros2_control, MoveIt config
`dynamixel_hardware_interface`, `robotis_interfaces`	ament	–	실물 하드웨어 의존성
`vla_sim`	ament_cmake	`worlds/vla_table.sdf`, `launch/sim.launch.py`, `config/omx_f_sim_controllers.yaml`, `scripts/gen_markers.py`	시뮬 월드, 마커 모델, 컨트롤러 게인
`vla_interfaces`	rosidl	`Block.msg`, `BlockArray.msg`, `PickPlace.action`, `ExecuteCommand.action`	인터페이스
`vla_perception`	ament_python	`table_geometry.py`, `perception_node.py`	마커 → 호모그래피 → 블록
`vla_executor`	ament_python	`omx_ik.py`, `executor_node.py`	IK + MoveIt + 그리퍼
`vla_planner`	ament_python	`llm_plan.py`, `planner_node.py`, `cli.py`	LLM, 계획 해석·검증, CLI
`vla_calibration`	ament_python	`intrinsics.py`, `colors.py`, `table.py`, `wrist.py`, `common.py`	캘리브레이션 도구 4종
`vla_bringup`	ament_cmake	`launch/sim_vla.launch.py`	전체 실행, `calib_dir` 로 캘리브레이션 결과 적용

2.3 인터페이스

`vla_interfaces/msg/Block` : `id` (예 `red_0`), `color`, `x`, `y` [m, link0], `yaw` ∈ [−π/4, π/4), `u`, `v` (픽셀), `area_px`

`vla_interfaces/msg/BlockArray` : `header` , `table_calibrated` , `blocks[]`

`vla_interfaces/action/PickPlace`

- Goal: `pick_x` , `pick_y` , `pick_yaw` , `pick_z_offset` , `place_x` , `place_y` , `place_yaw` , `place_z_offset`
- Result: `success` , `message`
- Feedback: `stage`

`vla_interfaces/action/ExecuteCommand`

- Goal: `command`
- Result: `success` , `message` , `plan_json`
- Feedback: `stage`

토픽/액션	타입	발행/서버
`/vla/blocks`	BlockArray	perception
`/vla/debug_image`	sensor_msgs/Image	perception (구독자가 있을 때만)
`/vla/command`	std_msgs/String	planner 구독
`/vla/status`	std_msgs/String	planner
`/execute_command`	ExecuteCommand	planner
`/pick_place`	PickPlace	executor

2.4 좌표계

- 기준: `link0` (OMX-F 베이스), +x 로봇 전방(먼 쪽), +y 로봇 왼쪽, 테이블 z = 0.
- 시뮬에서는 world = link0 (로봇을 원점에 spawn).
- 마커 중심 (ID: x, y): 0: (0.05, 0.20), 1: (0.33, 0.20), 2: (0.33, −0.20), 3: (0.05, −0.20). ArUco `DICT_5X5_100` , 한 변 5 cm (perception `aruco_dictionary` 파라미터, 시뮬 마커는 `gen_markers.py` 의 `DICT_NAME`).
- 탑 카메라: (0.19, 0, 0.75), 아래를 향함, 1280×960, HFOV 0.9 rad.
- 블록: 25 mm 정육면체, 20 g, μ = 2.0.

3. 인식 (`vla_perception`)

3.1 호모그래피

마커 4개가 모두 보이면 각 마커 코너 4점의 평균(중심) 픽셀 p_i와 테이블 좌표 P_i로 `cv2.findHomography` 를 호출해 H(pixel → table)를 구한다. 팔이 마커를 가리는 동안에는 마지막으로 유효했던 H를 유지한다. 카메라 왜곡 계수 D가 있으면 `cv2.undistortPoints(P=K)` 로 점을 이상 핀홀 좌표로 바꾼 뒤 사용한다(실물 대비).

3.2 카메라 중심 역산

평면 → 이미지 호모그래피 $H^{-1} = K[r_1\ r_2\ t]$ 관계에서:

1. $M = K^{-1}H^{-1}$, 스케일 $s = 2/(\lVert m_1 \rVert + \lVert m_2 \rVert)$로 정규화. $M_{33} < 0$이면 부호 반전.
2. $R = [r_1\ r_2\ r_1 \times r_2]$, SVD로 직교화 ($R \leftarrow UV^{\top}$).
3. 카메라 중심 $C = -R^{\top}t$.

시뮬 결과는 C = (0.190, −0.000, 0.750)으로 정확히 일치했다.

3.3 높이 시차 보정

호모그래피는 z = 0 평면 기준이므로, 높이 h인 블록 윗면은 카메라에서 바깥쪽으로 밀려 보인다. 광선을 따라 보정한다.

$$ \mathbf{x}_h = C_{xy} + (\mathbf{x}_0 - C_{xy})\,\frac{C_z - h}{C_z} $$

h = 25 mm, C_z = 0.75 m에서 중심으로부터 0.2 m 떨어진 블록의 보정량은 약 7 mm다.

3.4 블록 검출

1. 마커 중심 다각형(ROI, `roi_margin`으로 확장 가능)을 이미지로 역투영해 마스크를 만든다.
2. HSV 범위별 `inRange`, open/close 모폴로지(5×5), 외곽 윤곽선, 면적 필터(150–20000 px).
3. 윤곽선의 모든 점을 언디스토트 → 호모그래피 → 높이 보정한 뒤, **테이블 좌표계에서** `minAreaRect`로 중심과 yaw를 구한다. yaw는 정사각형 대칭이므로 mod $\pi/2$.
4. ID 부여: 색별로 베이스에서 가까운 순 `color_k`.

색	HSV 범위 (H: 0–179)
red	[0,120,70]–[8,255,255] ∪ [170,120,70]–[179,255,255]
green	[40,100,60]–[85,255,255]
blue	[100,120,60]–[130,255,255]
yellow	[20,100,100]–[35,255,255]

3.5 구현 메모

- `cv_bridge`를 쓰지 않고 `sensor_msgs/Image`를 numpy로 직접 변환한다(`rgb8`, `bgr8`, `mono8`, `yuyv`). 시스템에 pip OpenCV 5.0 / numpy 2.5가 설치되어 있어 numpy 1.x로 빌드된 바인딩과 ABI가 충돌하기 때문이다.
- 시뮬 결과(GT 대비): blue 2.1 mm, red 1.6 mm, green 0.9 mm, yellow 0.5 mm, yaw 오차 0°(초록 0.4 rad 회전 포함).

4. 동작 (`vla_executor`)

4.1 OMX-F 기구학 (URDF 기준)

관절	축	부모 기준 원점
joint1	z (yaw)	(−0.01125, 0, 0.034)
joint2	y (pitch)	(0, 0, 0.0635)
joint3	y	(0.0415, 0, 0.11315)
joint4	y	(0.162, 0, 0)
joint5	x (roll)	(0.0287, 0, 0)
end_effector_link	fixed	(0.09193, −0.0016, 0)
gripper_joint_1/2	z, mimic −1	(0.0295, ±…, 0)

MoveIt config는 `position_only_ik: True` (KDL)라서 그리퍼가 아래를 향한다는 보장이 없다. 그래서 **top-down 전용 닫힌 형 IK**를 직접 구현했다.

4.2 Top-down IK (`omx_ik.top_down_ik`)

목표 (x,y,z), 공구축이 −z를 향하도록:

1. $q_1 = \text{atan2}(y, x - x_{j1})$, $r = \text{//}(x - x_{j1}, y)\text{//}$
2. 손목점(joint4) = 공구 끝 바로 위: $u = r$, $v = z + L_{tool} - z_{sh}$ ($L_{tool} = 0.12063$, $z_{sh} = 0.0975$)
3. 2-링크($L_1 = 0.1205$, $\beta = \text{atan2}(0.11315, 0.0415)$, $L_2 = 0.162$) 엘보업 해: $\cos\gamma = (u^2 + v^2 - L_1^2 - L_2^2)/(2L_1L_2)$, $\gamma = -\arccos(\cdot)$, $\theta_a = \text{atan2}(v, u) - \text{atan2}(L_2\sin\gamma, L_1 + L_2\cos\gamma)$
4. $q_2 = \beta - \theta_a$, $q_3 = -(\theta_a + \gamma) - q_2$, $q_4 = \pi/2 - q_2 - q_3$
5. 손목 roll: $q_5 = \text{wrap}(q_1 + \pi/2 - \psi)$, 정사각형 대칭으로 $[-\pi/4, \pi/4]$로 접는다 (ψ = 블록 yaw).

검증: TF와 FK가 0.1 mm 이내로 일치. 테이블 높이 기준 최대 도달 반경은 약 0.25 m.

4.3 Pick & place 시퀀스

1. 입력 검증: 모든 IK 해를 움직이기 전에 계산. 반경 [0.09,0.25] m 밖이거나 해가 없으면 즉시 abort.
2. gripper open → pre-grasp (z + 0.06) → grasp (`grasp_z` + pick_z_offset) → gripper close → lift → **verify grasp (손목 카메라, § 4.5)** → pre-place → place (`place_z` + place_z_offset, 쌓기면 + `stack_clearance`) → release (`gripper_release`까지만 열기) → retreat → observe
3. 각 팔 동작은 `/move_action` (MoveGroup)에 **joint constraint 목표**로 보내고, 파이프라인은 `pilz_industrial_motion_planner` / `PTP` (결정적 관절공간 보간)를 쓴다. 충돌 검사와 궤적 실행은 MoveIt이 담당한다.
4. 동작마다 **settle 대기**: `/joint_states` 속도가 0.02 rad/s 미만으로 0.2 s 유지될 때까지(최대 3 s).
5. observe 자세 (0, −1.57, 1.57, 1.57, 0): 팔을 접어 카메라 시야를 비운다. 노드 시작 시에도 1회 실행한다 (`observe_on_start`).

4.4 주요 파라미터 (`config/executor.yaml`)

파라미터	값	비고
`grasp_z`	0.014	집을 때 end_effector_link 높이
`place_z`	0.014	놓을 때 높이 (+ 쌓기 오프셋 0.025 × 층)
`approach_height`	0.06	접근·후퇴 높이
`gripper_open` / `gripper_closed`	1.0 / 0.05 rad	25 mm 블록 접촉각 ≈ 0.18
`gripper_release`	0.35 rad	놓을 때는 이 각도까지만 열기
`stack_clearance`	0.004 m	쌓을 때 추가 놓기 높이
`wrist_roi` / `wrist_min_fill`	[0.40, 0.40, 0.65, 0.58] / 0.6	손목 카메라 집기 확인
`velocity_scaling` / `acceleration_scaling`	0.3 / 0.3	
`min_reach` / `max_reach`	0.09 / 0.25 m	
`settle_velocity` / `settle_timeout`	0.02 rad/s / 3 s	

4.5 손목 카메라 집기 확인

목적: lift 직후 블록이 실제로 집게 안에 있는지 확인한다. 시뮬에서는 그리퍼 관절각이나 stall 플래그로는 블록 유무를 구분할 수 없었다(§ 8 #10).

카메라 (시뮬): `sim.launch.py`가 URDF 문자열에 `wrist_camera_link`를 추가한다. 부모는 `link5`, 위치 (0.02, 0, −0.035), 피치 −0.46 rad로 손가락 판 옆 뒤쪽에서 손끝을 내려다본다. 320×240, HFOV 1.2, 10 Hz. 토픽은 `/wrist_camera/image_raw`, `/wrist_camera/camera_info`. URDF의 `end_effector_link` 빨간 표시 박스(1 cm)가 손가락 사이에 보여 빨간 블록 오탐을 일으키므로, 시뮬 URDF에서 해당 visual을 제거한다.

판정 (`grasp_check.grasp_fill`):

1. lift 후 settle이 끝난 뒤 들어온 새 프레임을 기다린다 (`wrist_timeout` 2 s).
2. ROI = 이미지 비율 [0.40, 0.40, 0.65, 0.58]. 손끝 바로 아래, 들린 블록이 차지하는 영역이다.
3. ROI 안에서 `pick_color`의 HSV 마스크 비율(fill)을 구한다. 색이 비어 있으면 모든 색 중 최대값. HSV 표는 perception과 공유한다(`vla_perception.table_geometry.DEFAULT_HSV`).
4. fill ≥ `wrist_min_fill` (0.6)이면 통과, 아니면 `GraspError`: 집게를 열고 observe 자세로 간 뒤 abort하고, 메시지에 측정 비율을 담는다.
5. `/vla/wrist_debug`에 ROI와 판정을 그린 이미지를 발행한다.

판정 근거: 들린 블록은 카메라에서 3–4 cm 거리라 ROI를 거의 채운다. 놓친 블록은 테이블에 남아 6 cm 이상 아래에 작게 보이고, 닫힌 손가락에 가려 ROI 밖에 있다.

시나리오	fill	판정
정상 집기 (빨강·초록 회전·노랑, 반복 8회)	85–99%	통과
3.5 cm 어긋난 위치에서 집기 (블록이 끌려 들어옴)	85%	통과 (실제로 잡힘)
빈 자리 집기	0%	실패
블록 위 1.5 cm / 3 cm에서 헛잡음	0% / 0%	실패
다른 색 블록을 집음 (노랑을 집고 초록으로 지정)	0%	실패

실물 적용: 손목 카메라가 장착되면 `wrist_image_topic`, `wrist_roi`, `wrist_min_fill`을 실측 영상으로 다시 맞춘다. 카메라가 없으면 `wrist_check: false`. 이미지가 안 들어올 때의 동작은 `wrist_required`로 정한다(true면 실패, false면 경고 후 진행).

5. 언어 계획 (`vla_planner`)

5.1 스키마 제약 디코딩

요청마다 **현재 검출된 블록 ID로 JSON 스키마를 동적으로 생성**해 Ollama `format`으로 전달한다. Ollama는 이 스키마를 문법(grammar)으로 바꿔 디코딩을 제한한다.

```
{ "feasible": bool, "reason": str,
  "steps": [ { "pick":  enum[<detected ids>],
               "place": enum[ center | left | right | front | back |
                              front_left | front_right | back_left | back_right |
                              on_top:<id> | left_of:<id> | right_of:<id> |
                              front_of:<id> | back_of:<id> ] } ]  (max 4) }
```

`place` 문자열은 `parse_place()`가 `place_mode` / `region` / `target` / `direction`으로 분해한다. 시스템 프롬프트에는 방향 규약(로봇 시점), 한국어 어휘 매핑(앞/뒤/왼쪽/오른쪽/위에/옆, 색 이름), few-shot 예시 4개, 불가능한 요청을 거부하는 규칙이 들어 있다. `temperature` = 0.

5.2 계획 해석·검증 (`resolve_plan`)

- 영역 좌표: center (0.18, 0), left/right (0.17, ±0.12), front (0.23, 0), back (0.12, 0), 대각 (0.21, ±0.10) / (0.12, ±0.10)
- relative: 대상 블록 + 방향 × 0.05 m
- on_top: 대상 xy·yaw, 층 = 대상 층 + 1, `place_z_offset` = 층 × 0.025
- 단계 간 **가상 월드 상태**를 갱신해 다단계 명령(쌓은 블록 위에 또 쌓기 등)을 처리한다.

- 거부 조건: 미검출 ID, 반경 밖, 다른 블록과 3.5 cm 이내, 위에 블록이 있는 블록 집기, 자기 자신 위에 쌓기, `feasible = false`
- 층 기억: 카메라로는 높이를 알 수 없으므로, 플래너가 직접 쌓은 블록의 (색, xy, 층)을 기억해 다음 명령의 pick 높이에 반영한다.

5.3 실행 후 검증

각 단계 후 새 `/vla/blocks` 프레임에서:

- 옮긴 색의 블록이 목표 3 cm 이내에 있는지 확인 (`verify_tolerance`)
- 쌓기일 경우, 아래 블록이 1.5 cm 넘게 떨어진 곳에서 보이면 무너진 것으로 판단

5.4 LLM 평가 (명령 10개, gemma4:e2b)

버전	정답	평균 지연	실패 사례
v1 (자유 문자열 + enum region)	8/10	5.3 s	region 누락 + 단계 4회 중복, “오른쪽으로 치워”를 relative로 해석
v2 (동적 enum 스키마 + few-shot)	**10/10**	**3.1 s**	–

이 외에 end-to-end 중 v1이 `"pick": "id"` (플레이스홀더 문자열)를 출력한 사례도 있었으며, v2에서 구조적으로 불가능해졌다.

6. 시뮬레이션 (`vla_sim`)

- Gazebo Harmonic (gz-sim 8.15), 물리 2 ms, Sensors 시스템(ogre2)
- `gen_markers.py` : ArUco PNG(흰 여백 포함)와 plane visual 모델 생성. **텍스처 파일명은 마커마다 달라야 한다**(같은 `marker.png` 이름을 쓰면 gz 텍스처 캐시가 재사용해 모든 마커가 ID 0으로 보인다).
- `sim.launch.py` : ROBOTIS `omx_f_gazebo.launch.py` 를 바탕으로 월드, 리소스 경로, 카메라 브리지를 교체. URDF의 컨트롤러 파라미터 경로를 `vla_sim/config/omx_f_sim_controllers.yaml` 로 치환한다.
- **`gz_ros_control.position_proportional_gain: 1.0`** (기본 0.1): gz_ros2_control은 position 명령을 이 P 게인의 velocity 명령으로 바꾼다. 기본값에서는 중력 때문에 최대 0.038 rad의 정상상태 오차가 생겨, 그리퍼가 블록 위에 걸리고 MoveIt 시작상태 검증에 실패했다.

7. 테스트 결과 상세

7.1 인식 정확도

§ 3.5 참고. 최대 2.1 mm.

7.2 관절 추종

게인	명령 z 0.060 / 0.030 / 0.015 → 실측 z	최대 관절 오차
0.1	0.0624 / 0.0322 / 0.0163 (그리퍼 닫힘)	0.038 rad
1.0	0.0600 / 0.0300 / 0.0150	< 0.001 rad

7.3 반복 시험 (3회, `/pick_place` 직접 호출)

시험	결과
red (0.18, 0.08) → (0.15, −0.03)	3/3, 4.7 / 4.3 / 5.4 mm (일관된 −x, −y 편향)
green (yaw 0.4) → red 위	3/3 집기 성공, 쌓기 2/3

7.3.1 손목 확인·놓기 개선 후 반복 시험

시험	결과
yellow → 회전된 green (yaw 0.4) 위, 3회	3/3
blue → red (0.18, 0.08) 위, 3회	2/3 (`gripper_release` 적용 전 1/3)
red 이동 + green(회전) → red 위, 3회	이동 3/3 (4.3–5.7 mm), 쌓기 3/3, 아래 블록 밀림 없음
손목 확인 fill	통과 6/6 (85–92%)

7.4 최종 end-to-end (`/execute_command`)

명령	결과	오차	소요
빨간 블록을 가운데로 옮겨	성공	5.2 mm	32 s (LLM cold 15 s)
파란 블록을 빨간 블록 위에 쌓아줘	성공, z = 0.037	6.1 mm	18 s
노란 블록을 왼쪽 앞으로 옮겨	성공	3.5 mm	19 s
초록 블록을 노란 블록 뒤에 놔	성공	12.6 mm	21 s
보라색 블록을 가운데로 옮겨	거부 (feasible = false)	–	1 s

7.5 단위 테스트

`vla_planner/test/test_resolve_plan.py` : 15개 통과 (region / relative / 다단계 쌓기 층 추적 / 묻힌 블록 / 기억된 층 / 거부 4종 / infeasible / parse_place 4종 / 스키마 enum).

8. 문제 해결 기록

#	증상	원인	조치
1	colcon 빌드 시 `No module named catkin_pkg`	활성화된 venv의 python이 CMake에 잡힘	venv 해제, `-DPython3_EXECUTABLE=/usr/bin/python3`
2	빌드 exit 137	24코어 병렬 컴파일로 15 GB RAM 초과	`--parallel-workers 2`, `MAKEFLAGS=-j4`
3	마커 4개가 모두 ID 0	gz 텍스처 캐시가 파일명으로 공유	마커별 파일명
4	카메라 영상이 프레임마다 바뀜	이전 gz 서버가 같은 gz 토픽에 계속 발행	프로세스 정리 후 재실행
5	집기 높이가 1 cm 높음, 블록 윗부분만 잡힘	gz position P 게인 0.1에서 중력 처짐	게인 1.0
6	MoveIt CONTROL_FAILED: start point deviates	컨트롤러 성공 보고 직후 팔이 아직 움직이는 중	settle 대기
7	그리퍼 0까지 닫으면 블록이 튀어나감	(게인이 낮던 시점) 얕게 잡힌 상태에서 과도한 조임	게인 수정 후 closed = 0.05로 재조정
8	쌓을 때 아래 블록이 밀려남	이동 중 블록이 기울어져 2 cm 어긋남 (그립 약함, closed = 0.12)	closed = 0.05
9	실패를 성공으로 보고	결과 검증 없음	카메라 기반 사후 검증
10	그리퍼 stall 기반 miss 감지 오탐	시뮬에서 블록 유무와 무관하게 각도·플래그가 같음	제거, 카메라 검증으로 대체
11	LLM이 `"id"`, region 누락	스키마 제약 약함	동적 enum 스키마
12	회전 블록 위 쌓기에서 아래 블록이 0.46 m까지 튕겨 나감, 그런데 검증 통과	블록끼리 맞닿는 높이에서 놓아 겹침 반발. 아래 블록이 카메라 영역 밖으로 나가 "가려짐"으로 오인	`stack_clearance` 4 mm (검증 한계는 §9)
13	놓는 순간 블록과 탑이 함께 날아감	깊게 조인 상태(0.05)에서 1.0까지 한 번에 열며 겹침이 폭발적으로 해소	`gripper_release` 0.35까지만 열기
14	손목 카메라에 빨간 박스가 항상 보임	URDF의 EE 표시 visual	시뮬 URDF에서 제거
15	캘리브레이션 결과 YAML을 ROS가 못 읽을 수 있음	PyYAML이 같은 리스트를 `&id001` / `*id001` 별칭으로 저장	별칭 없는 Dumper
16	table 캘리브레이션 맞춤 잔차 88 mm	놓기 오차가 누적됨(명령 위치에서 다음 블록을 집어서 오차가 이어짐), 회전된 블록을 yaw 0으로 집다 실패	관측 yaw로 집기, drift 검사·재시도
17	호모그래피 방식 마커 오차 12 mm	좁은 작업 반경에서 넓은 마커 범위로 외삽	2D 강체 변환(자유도 3) + 이상치 제거

9. 한계와 남은 과제

정확도

- 놓는 위치에 일관된 편향(≈ −4 mm x, −3 mm y)이 있다. grasp 시 블록이 손가락 사이에서 미세하게 미끄러지는 것으로 추정되며, 보정 오프셋 파라미터로 처리할 수 있다.
- relative 배치(12.6 mm)는 회전된 블록을 놓을 때 yaw 정렬과 가장자리 접촉 영향으로 보인다.

쌓기 신뢰성 (8/9)

- 남은 실패는 놓는 순간 블록끼리의 접촉에서 생긴다. 내려놓기 속도를 낮추고 release 전에 짧게 정지하는 것을 검토한다.
- 탑 카메라로는 "아래 블록이 가려짐"과 "카메라 영역 밖으로 튕겨 나감"을 구분할 수 없다. retreat 높이에서 손목 카메라로 위 블록의 겉보기 크기(쌓였으면 2.5 cm 더 가까움)를 재는 쌓기 확인을 추가할 예정이다.

그리퍼 피드백

- 집기 성공 여부는 손목 카메라로 확인한다(§ 4.5). 실물에서는 Dynamixel Present Current를 보조 신호로 추가할 수 있다.

인식 한계

- 높이를 알 수 없다(Depth 없음). 쌓인 블록의 층은 플래너 기억에만 의존하며, 사람이 손으로 쌓은 블록은 알지 못한다.
- 같은 색 블록이 여러 개이면 ID가 거리 순으로 정해져서, 블록을 옮기면 ID가 바뀔 수 있다. 현재 시나리오는 색당 1개다.
- HSV 고정 범위라 조명 변화에 취약하다. 실물에서 재튜닝이 필요하다.

LLM

- 영역 좌표가 고정값이다. 테이블 크기가 달라지면 `DEFAULT_REGION_XY` 를 수정해야 한다(파라미터화 예정).
- 이미지 입력(VLM)은 아직 쓰지 않고, 텍스트 장면 기술만 사용한다.

10. 캘리브레이션 (`vla_calibration`)

10.1 구성과 순서

결과는 `~/dev_ws/robot/calib/` 에 ROS 2 params YAML로 저장된다(노드별 섹션, 측정한 값만 포함). `sim_vla.launch.py calib_dir:=<dir>` 이 기본 설정 뒤에 덮어 적용한다. YAML은 ROS 파서가 읽지 못하는 앵커/별칭을 쓰지 않도록 저장한다.

순서	도구	입력	출력 → 적용 대상
1	`intrinsics`	체커보드 영상 (`/camera/image_raw`)	`camera_info.yaml` → usb_cam `camera_info_url`
2	`colors`	색별 블록 1개씩 놓인 탑뷰 영상	`colors.yaml` → perception · executor `colors` , `hsv.*`
3	`table`	로봇 + 블록 1개 + `/vla/blocks`	`table.yaml` → perception `marker_positions`
4	`wrist`	로봇 + 블록 1개 + 손목 카메라	`wrist.yaml` → executor `wrist_roi` , `wrist_min_fill`

순서가 중요하다. table은 집기 확인과 블록 검출에 색 범위를 쓰고, wrist는 색 범위와 정확한 블록 위치를 쓴다. 로봇을 쓰는 도구(table, wrist)는 `PickPlaceExecutor(serve=False)` 를 상속해 executor와 같은 동작 코드(`run_pick_place` ,

`move_joints`, `gripper`)를 쓰고, `--executor-config`로 실행 중인 executor와 같은 파라미터를 읽는다.

10.2 카메라 내부 파라미터 (`intrinsics`)

- `findChessboardCorners` + `cornerSubPix`. 보드 자세 특징(정규화 중심, 크기, 기울기)이 기존 샘플과 0.06 이상 다를 때만 수집해 비슷한 자세가 몰리지 않게 한다.
- `cv2.calibrateCamera` (plumb_bob) 결과를 ROS camera_info YAML로 저장하고, 뷰별 재투영 오차와 전체 RMS를 출력한다.
- 시뮬 검증: `vla_sim/checkerboard_sweep.py`가 9×6 보드(30 mm)를 무작위 자세로 움직인다. 결과는 fx 1325.28 / fy 1325.38 / cx 640.11 / cy 480.65 (Gazebo camera_info: 1324.90 / 1324.90 / 640 / 480, **fx +0.03 %**), RMS 0.22 px, 왜곡 계수 ≈ 0 (시뮬 카메라는 무왜곡).

10.3 블록 색 (`colors`)

1. 마커 4개 중심의 볼록 다각형에서 마커 영역을 뺀 마스크 안에서 S ≥ 80, V ≥ 50인 연결 영역(≥ 150 px)을 찾는다.
2. 영역별 원형 중앙 hue(0/179 경계 처리)를 요청 색의 기준 hue(red 0, yellow 28, green 60, blue 115 등)에 매칭한다. 오차가 20을 넘으면 버린다.
3. 15프레임의 픽셀을 모아 hue는 중심 기준 1–99 백분위 ± 6, S/V 하한은 2 백분위 − 40/50(최소 60/40)으로 정한다. 경계를 넘으면 두 구간으로 나눈다.
4. 색 간 hue 겹침을 경고하고, 수집 픽셀 재현율을 출력한다.

시뮬 결과: red [0–6] ∪ [174–179], green [54–66], blue [114–126], yellow [24–36], 재현율 98–100 %, 겹침 없음.

10.4 마커 ↔ 로봇 (`table`)

문제: perception은 `marker_positions` (로봇 기준 마커 중심)를 그대로 믿는다. 실물에서 마커 사이 거리는 자로 정확히 잴 수 있지만, 그 사각형이 로봇 베이스 기준 어디에 있는지는 재기 어렵다.

방법:

1. 블록 1개를 로봇이 직접 12곳(작업 반경 안 격자)에 놓는다. 각 지점은 직전 지점에서 집는다(직전에 관측한 yaw 사용).
2. 놓을 때마다 observe 자세에서 `/vla/blocks`로 블록 위치(현재 마커 좌표계 기준)를 6프레임 평균한다.
3. 명령 위치에서 `--max-drift` (3 cm) 넘게 벗어나면 미끄러진 것으로 보고 제외하고, 다음 집기는 관측 위치에서 한다. 집기 확인이 실패하면 다시 관측해 1회 재시도한다.
4. 관측점 → 명령점의 2D 강체 변환을 Umeyama(SVD)로 맞춘다(`--fit-scale`이면 균일 배율 포함). 잔차가 중앙값의 2.5배와 5 mm를 넘는 점은 최대 25 %까지 반복 제거한다.
5. 같은 변환을 현재 `marker_positions`에 적용해 저장한다. 출력: 변환(회전, 이동), 맞춤 잔차, leave-one-out 오차, 마커별 이동량.

자유도 8 호모그래피를 쓰지 않은 이유: 처음 구현은 블록 점 → 픽셀 호모그래피를 맞추고 카메라 중심을 역산해 마커로 외삽했다. 작업 반경(x 0.13–0.235 m)이 마커 범위(x 0.05–0.33, y ±0.2)보다 좁아 외삽 오차가 컸다(마커 오차 최대 12 mm, 카메라 위치 오차 7 cm). 마커 간 거리를 알고 회전·이동(자유도 3)만 맞추면 조건이 훨씬 좋다.

시뮬 검증: perception을 일부러 틀어진 마커 배치(2° 회전 + (+12, −8) mm 이동)로 실행한 뒤 캘리브레이션을 돌렸다.

항목	값
사용 점	9 (10번째 지점 집기 실패 → 9점으로 맞춤)
오차 (캘리브레이션 전 → 후)	평균 9.9 mm → 맞춤 잔차 3.1 mm (최대 5.6), leave-one-out 3.8 mm (최대 7.2)
회전	**−2.04°** (넣은 값 −2.0°)
마커 y 좌표 (실제 대비)	0.0–0.2 mm
마커 x 좌표 (실제 대비)	+8.6 ~ +8.9 mm (아래 설명)

이 캘리브레이션은 카메라 좌표계를 **로봇이 실제로 블록을 놓는 위치**에 맞춘다. 시뮬의 OMX-F는 블록을 명령 위치보다 x 방향으로 약 6–9 mm 짧게 놓는 경향이 있고(잡은 블록이 손가락 판 길이 방향으로 치우침), 이 편향이 변환에 흡수된다. 결과를 적용한 end-to-end 시험(3개 명령, 쌓기 포함)에서 카메라가 본 위치와 목표의 차이는 0–1 mm, Gazebo 절대 좌표와의 차이는 9–10 mm였다. 쌓기·옆에 놓기처럼 블록 기준인 작업에는 일관성이 중요하므로 이 방식을 택했다.

10.5 손목 카메라 집기 확인 (`wrist`)

1. `/vla/blocks` 에서 지정 색 블록을 찾아 세 상태의 손목 영상을 각각 5장 찍는다.
   - held: 집고 들어 올린 상태
   - empty: 블록을 내려놓은 뒤 같은 높이에서 빈 채로 닫은 상태
   - miss: 블록 2 cm 위에서 닫고 들어 올린 상태(블록은 테이블에 남음)
2. ROI: held 색 마스크 5장의 교집합에서 가장 큰 연결 영역의 bbox를 12 % 줄인다.
3. 기준값: (held 최소 + empty·miss 최대) / 2. 둘 사이 여유가 25 % 미만이면 경고하고 종료 코드 2로 끝낸다.
4. ROI와 판정을 그린 `wrist_calibration.png` 를 저장한다.

시뮬 결과: ROI [0.43, 0.348, 0.636, 0.648], held 94.9–100 %, empty·miss 5.7 %, 기준 50.3 %, 여유 89 %.

11. 실물 이전 계획

1. **하드웨어 bringup**: `ros2 launch open_manipulator_bringup omx_f.launch.py port_name:=/dev/ttyUSB*` , 이어서 `omx_f_moveit.launch.py`
2. **카메라**: `usb_cam` 로 `/camera/image_raw` 를 발행한다. `vla_calibration intrinsics` 로 `camera_info.yaml` 을 만들고 `camera_info_url` 로 지정한다. 왜곡 보정은 perception에 이미 구현되어 있다.
3. **마커**: 마커 중심 사이 거리를 자로 재서 `marker_positions` 초기값(대략적인 로봇 기준 위치 + 정확한 상대 배치)을 넣고, `colors` → `table` → `wrist` 순으로 캘리브레이션한다(§ 10).
4. **파라미터 재조정**: `block_height` , `grasp_z` / `place_z` , `gripper_closed` (전류 제한 고려), `use_sim_time: false`
5. **안전**: velocity scaling 0.1부터 시작하고, 작업 반경과 테이블 충돌 객체를 planning scene에 추가한다.
6. **launch 분리**: `vla_bringup/real_vla.launch.py` (sim과 같은 노드에 real 설정 파일)

12. 두 팔 확장 (left / right, 2026-10-16)

12.1 배경과 구성

실제 셋업은 OMX-F 2대다. - **배치:** 두 팔이 7.5 cm 높이 보드 위에 같은 방향(+x)으로 놓이고, 두 베이스 판 사이 간격은 18 cm다. - **작업면:** 보드보다 7.5 cm 낮은 바닥이다. - **인식:** 탑뷰 카메라 1대와 바닥 마커 4개(`DICT_5X5_100`)를 쓴다. - **명칭:** `left` / `right` 를 쓴다. ROBOTIS의 리더 모델 이름 `omx_l` 과 섞이지 않게 하기 위해서다.

구성	파일	내용
결합 URDF	`vla_dual_description/urdf/omx_dual.urdf.xacro`	`world` (바닥) → 보드(충돌 박스) → `left_` / `right_` `omx_f` 매크로 (ROBOTIS 재사용). 베이스 위치 인자 `base_separation` (기본 0.33) 또는 팔별 `*_x/_y/_yaw`. ros2_control 시스템 2개
시뮬 컨트롤러	`vla_dual_description/config/dual_sim_controllers.yaml`	`left/right_arm_controller` (JTC), `left/right_gripper_controller`, gz 위치 게인 1.0
MoveIt	`vla_dual_moveit_config`	그룹 `left_arm`, `right_arm`, `both_arms`, `left/right_gripper`. 팔 간 충돌 검사. Pilz PTP는 IK 솔버가 있는 단일 팔 그룹에서만 쓰고, `both_arms`는 OMPL을 쓴다
월드	`vla_sim/worlds/vla_floor.sdf`, `launch/dual_sim.launch.py`	바닥 마커 (0.10/0.40, ±0.40), 카메라 높이 1.0 m, 블록 4개, 보라색 트레이 (0.24×0.06 m, 양끝 25 mm 손잡이), 손목 카메라 2대
전체 실행	`vla_bringup/launch/dual_sim_vla.launch.py`	`*_dual.yaml` 설정으로 perception·executor·planner 실행

12.2 좌표계

`world` = 바닥(마커) 좌표계이고, z = 0이 작업면이다. perception은 `frame_id: world`로 블록을 발행한다. executor의 `Arm` 객체는 TF(`world → <arm>_link0`)로 베이스 pose (x, y, z = 0.075, yaw)를 한 번 읽어 두고, 목표를 팔 기준 좌표로 바꾼 뒤 기존 `top_down_ik`를 그대로 쓴다. 바닥(베이스 기준 z = −0.061)에서도 도달 거리는 0.28 m로 탁상 위와 같다(손목 높이가 어깨 기준으로 대칭이기 때문).

12.3 executor

- **팔 설정:** `arm_names`와 `arms.<name>.{joint_prefix, group, gripper_action, base_frame, wrist_topic}`으로 정한다. 단일 팔 설정(`['']`)과 호환된다.
- **팔 선택 (`choose_arm`):** `PickPlace.arm`이 비어 있으면, 집기·놓기 IK가 모두 되는 팔 중 집기 지점에 가까운 팔을 고른다. 팔을 지정했는데 닿지 않거나 어느 팔도 안 되면, 움직이기 전에 이유와 함께 실패한다.
- **동작 순서:** move_group 액션은 한 번에 한 목표만 받으므로 팔 동작을 하나씩 실행한다(노드 전체 잠금).
- **협동 운반 (`/carry_together`):**
  1. 손잡이 위치를 계산한다: 중심 ± (L/2 − 0.0125)·(cos ψ, sin ψ).
  2. 두 팔에 손잡이를 배정한다(도달 거리 합이 작은 쪽).
  3. 팔마다 차례로 접근 → 잡기 → 손목 카메라 확인을 한다.
  4. 트레이 경로(들기 4 cm → 이동 → 내리기)를 1 cm 간격으로 나누고, 각 점에서 두 손잡이 위치로 IK를 풀어 10관절 궤적을 만든다. 손목 roll은 90° 주기를 고려해 연속으로 맞춘다.
  5. 이 궤적을 `/execute_trajectory`로 한 번에 실행해 두 팔을 시간 동기로 움직인다.
  6. 동시에 놓고, 팔마다 수직으로 빠진 뒤 관찰 자세로 간다.

12.4 planner

- **스키마:** 두 팔이면 step에 `arm` enum(`auto` / `left` / `right`)을 추가하고, 프롬프트에 팔 어휘와 트레이 규칙을 넣는다.
- **`resolve_plan(arms=…)`:** 팔별 도달 범위를 검사한다.

- `auto` : 집기·놓기 모두 닿는 팔 중 가까운 팔을 쓴다.
- 그런 팔이 없으면 전달 지점을 찾는다. 조건은 두 팔 모두 닿을 것, x ≥ 0.10(보드에서 떨어질 것), 다른 물체와 5 cm 이상 떨어질 것이다. 도달 거리 여유를 크게 남기는 지점을 먼저 고르고(0.04 → 0.01 m), 그다음 이동 거리가 짧은 지점을 고른다. 찾으면 두 단계로 나눈다.
- 트레이(kind = tray)는 `carry` 동작이 된다. 시작·도착 모두 두 손잡이가 서로 다른 팔에 닿는지, 트레이를 선분으로 보고 다른 물체와 충돌하지 않는지 검사한다. 트레이 쌓기는 금지한다. 블록을 트레이에 놓는 경우(`on_top:<tray>` 또는 영역 점이 트레이 판 위, 손잡이에서 `min_clearance` 이상)에는 그 점에 `tray_plate` (8 mm)만큼 높여 놓는다. 2팔 월드 기본 배치에서 `center` (0.13, 0)는 그릇이 차지하고 있고 팔이 닿는 빈 자리는 14 cm 이상 떨어져 있어, 예전에는 "가운데로 옮겨"가 항상 거부됐다.

- **planner node:** 팔 베이스는 TF로 읽는다. 영역 좌표는 `region_names/x/y` 파라미터로 받는다. 트레이 결과는 카메라로 kind = tray인 물체를 찾아 확인한다.

12.5 시뮬 결과

시험	결과
MoveIt	팔별·양팔 계획 성공. 두 팔이 같은 점을 향하면 `left_link5` – `right_link7` 충돌로 거부
인식 (`world`)	블록 0.5–3.0 mm, 트레이 중심 2.5 mm, yaw 0°, 크기 0.242×0.058 m
팔 선택	왼팔 전용·오른팔 전용 블록 자동 배정, 잘못 지정한 팔 거부, 어느 팔도 안 되는 이동 거부
바닥 pick & place	오른팔 2.6–6.5 mm. 왼팔은 대부분 4–7 mm이고, 6회 중 2회 18–27 mm(집게 안 미끄러짐)
협동 운반	5회 중 4회 6.7–9.7 mm, 운반 중 기울기 1.9–6.2°. 1회는 트레이가 보드 쪽으로 끌려 89 mm(재현 안 됨, executor는 성공으로 보고. 자연어 경로에서는 카메라 검증이 잡음)
자연어	왼팔 지정 6 mm. 오른팔이 닿지 않는 명령 거부. 전달: 전달 지점이 처음엔 보드 가장자리(x 0.065)로 잡혀 계획 실패 → `handoff_x_min` 추가. 다음엔 도달 한계(0.236 m)에 잡혀 5 cm 오차 → 여유 우선 탐색 추가 → 성공(최종 11 mm). 그릇 운반 8.6 mm
팔 베이스 캘리브레이션	왼팔 (−0.008, 0.162, +0.9°) vs 실제 (0, 0.165, 0°). 로봇의 놓기 편향이 반영됨. 6점 사용(1점 이탈, 1회 재집기, 8번째 점에서 블록 분실)
손목 ROI (오른팔)	잡음 97.7–100 % / 빈 집게·헛잡음 0 %, 기준 48.8 %
단위 테스트	43개 통과 (planner 26, perception 4, executor 5, calibration 8)

12.6 한계와 다음 단계

- **공유 영역:** 두 팔이 함께 닿는 영역은 y ≈ 0, x 0.10–0.17 m 정도로 좁다. 트레이가 이 영역에 있으면 전달할 수 없다(planner가 이유를 말하고 거부).
- **시뮬 집게:** 집게 안에서 가끔 미끄러지거나 블록이 튕긴다. 기울기는 협동 운반 5회 중 2회만 목표(5° 미만)를 넘지 않았다. 운반 1회 실패는 원인을 찾지 못했다. 실물에서 다시 측정해야 한다.
- **실물 2팔:**
  - ID 11–16이 겹쳐서 U2D2 2개가 필요하다.
  - ROBOTIS 매크로의 하드웨어 서비스 이름이 겹친다.

    - 베이스 간 거리와 보드 가장자리 위치를 실측해 URDF 인자에 넣어야 한다.
- **범위 밖:** 두 팔 동시 독립 동작, 운반 중 MoveIt 충돌 검사(지금은 시작 상태만 검사).

부록 A. 실행 명령

```
        # 빌드
        deactivate 2>/dev/null; cd ~/dev_ws/robot; source /opt/ros/jazzy/setup.bash
        MAKEFLAGS=-j4 colcon build --symlink-install --parallel-workers 2 \
          --cmake-args -DPython3_EXECUTABLE=/usr/bin/python3

        # 실행 (모든 터미널)
        source ~/dev_ws/robot/install/setup.bash; export ROS_DOMAIN_ID=56
        ros2 launch vla_bringup sim_vla.launch.py   # headless:=true rviz:=false model:=... dry_run:=true
        ros2 run vla_planner cli                     # 한국어 명령 입력

        # 직접 호출
        ros2 action send_goal --feedback /execute_command vla_interfaces/action/ExecuteCommand "{command: '빨간
블록을 가운데로 옮겨'}"
        ros2 action send_goal /pick_place vla_interfaces/action/PickPlace "{pick_x: 0.18, pick_y: 0.08, place_x:
0.18, place_y: 0.0}"
        ros2 topic echo /vla/blocks

        # 캘리브레이션 (순서대로)
        ros2 run vla_calibration intrinsics --board 9x6 --square 0.03
        ros2 run vla_calibration colors --colors red green blue yellow
        ros2 run vla_calibration table --color red
        ros2 run vla_calibration wrist --color red
        ros2 launch vla_bringup sim_vla.launch.py calib_dir:=~/dev_ws/robot/calib
        ros2 run vla_sim checkerboard_sweep.py --poses 40    # 시뮬에서 체커보드 움직이기

        # 테스트 (src/ 에서)
        (cd vla_planner && python3 -m pytest -q test)
        PYTHONPATH=vla_executor:vla_perception:$PYTHONPATH python3 -m pytest -q vla_executor/test
        PYTHONPATH=vla_calibration:vla_executor:vla_perception:$PYTHONPATH python3 -m pytest -q
vla_calibration/test
```

부록 B. 환경

Ubuntu 24.04 (Linux 7.0) · ROS 2 Jazzy · MoveIt 2.12.4 · Gazebo Harmonic 8.15 · OpenCV 5.0 (pip) / numpy 2.5 · Ollama + `gemma4:e2b` (4.6 GB) · RTX 5080 Laptop 16 GB · RAM 15 GB · 24 cores